

프로그래밍과 AI

2026년 여름 비교과과목 바이브코딩 1

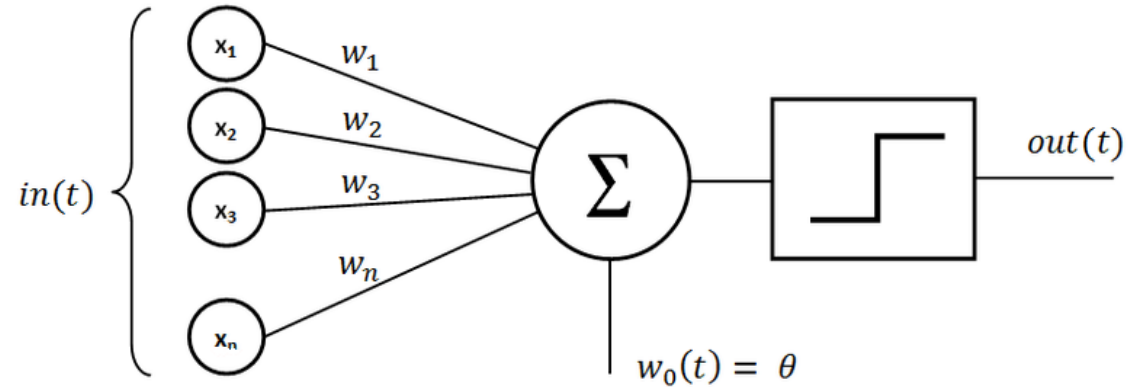
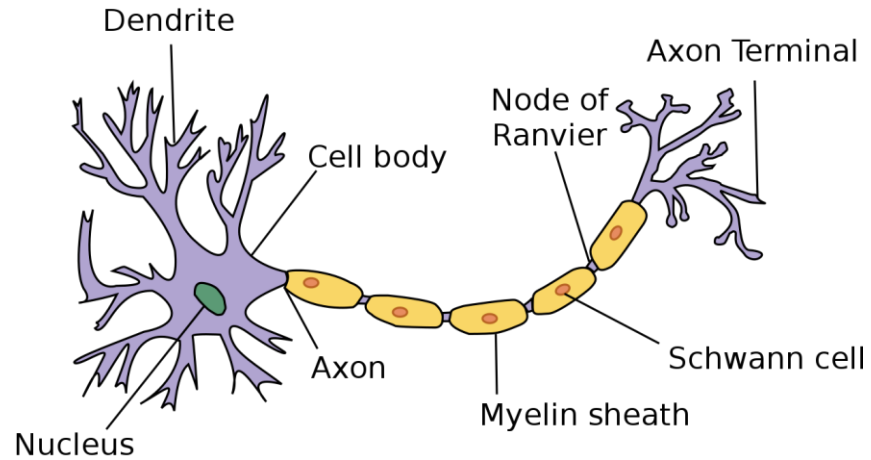
좋은 프로그래머란?

- ~ 1960년대 메모리 최적화
- ~ 1990년대 실행속도
- ~ 2010년대 구조/유지보수
- 현재 토큰사용량 최소화?

AI의 발전

- 1956년 휴리스틱
- 1960년 마크 1 퍼셉트론
- 1960년대 수학적 논리에 기반한 추론
- 1970년대 암흑기
- 1980년대 도메인 규칙에 기반한 추론
- 1990~2000년대 암흑기
- 2010년대 신경망 이론의 부활 (딥러닝)
- 2020년대 트랜스포머, 디퓨전 등 생성형 AI

신경망 기술 - 퍼셉트론

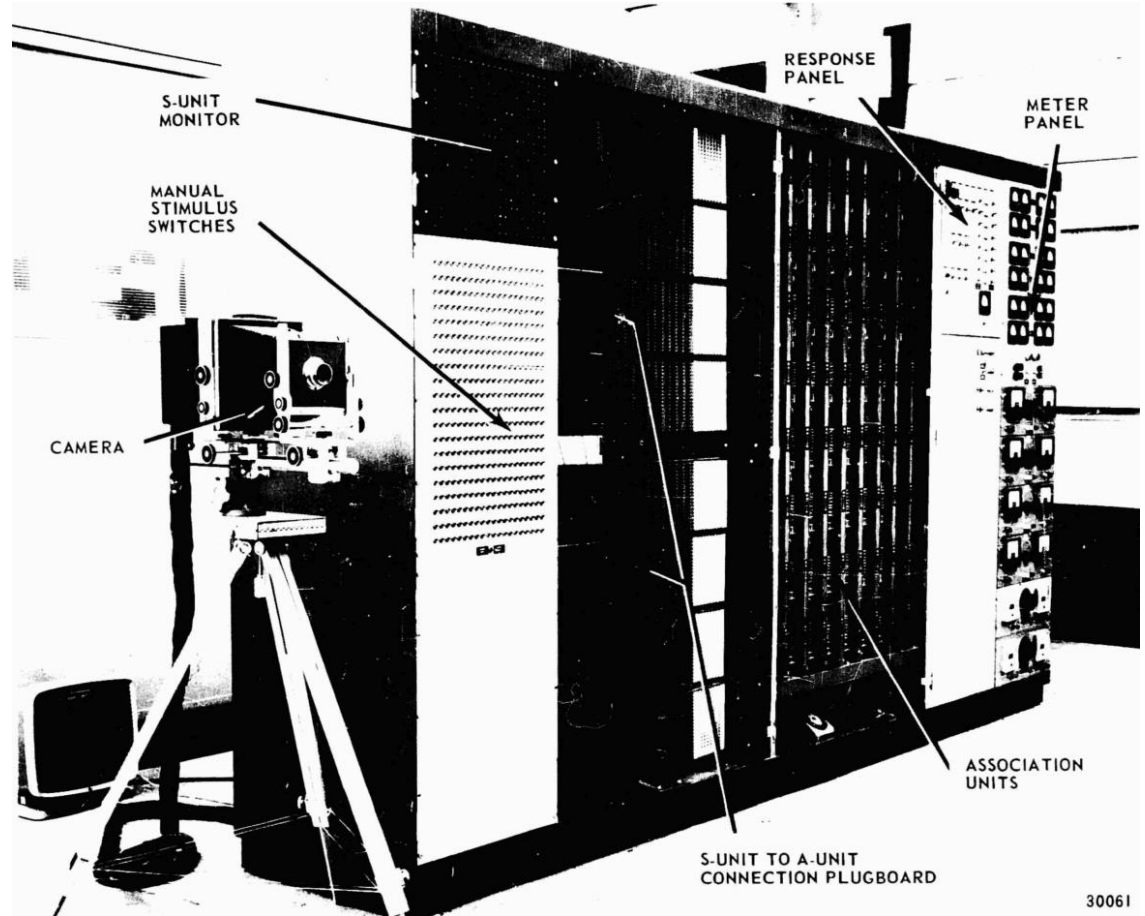


신경망 기술 - 마크 1 퍼셉트론

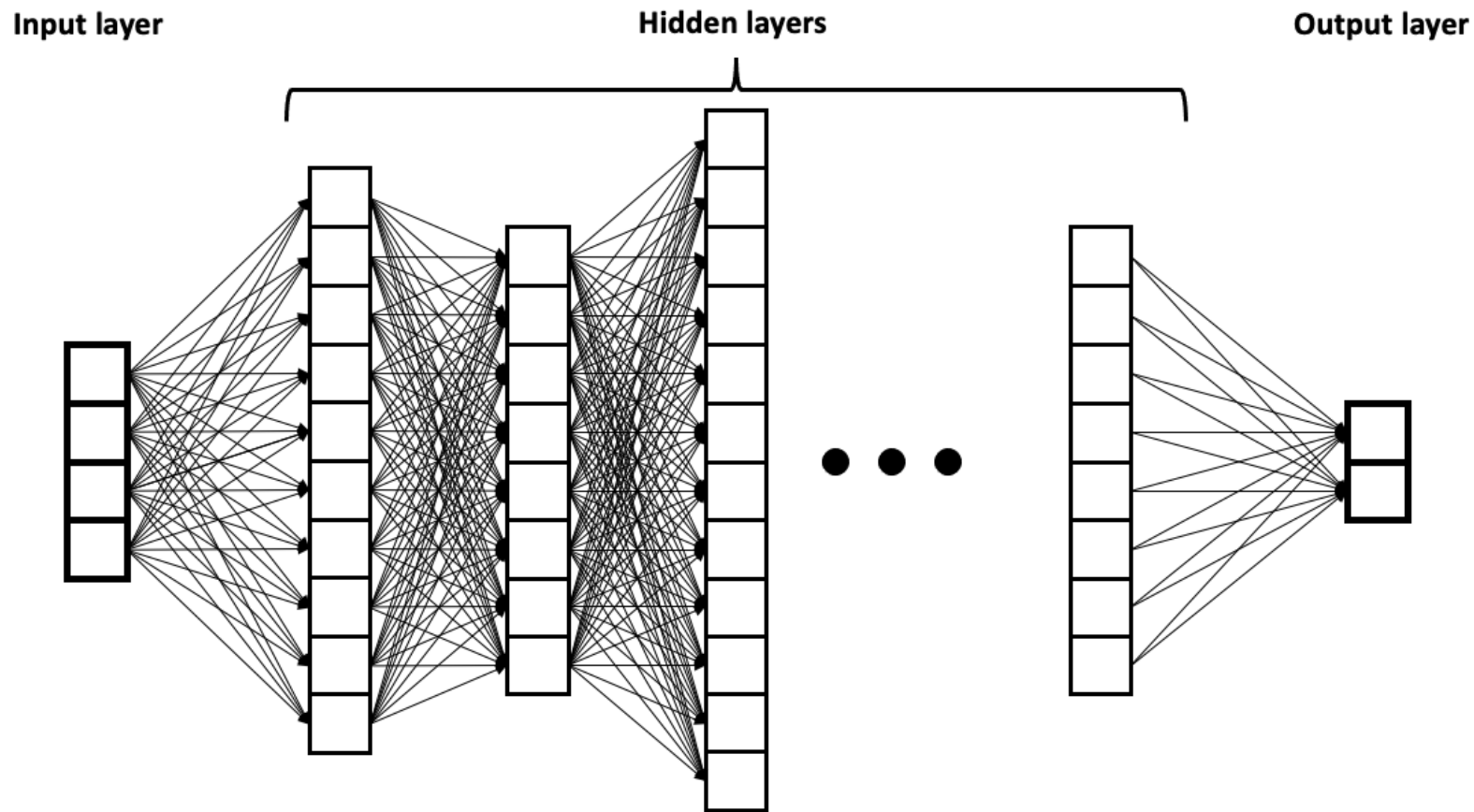
1957년에 설계

1960년에 완성

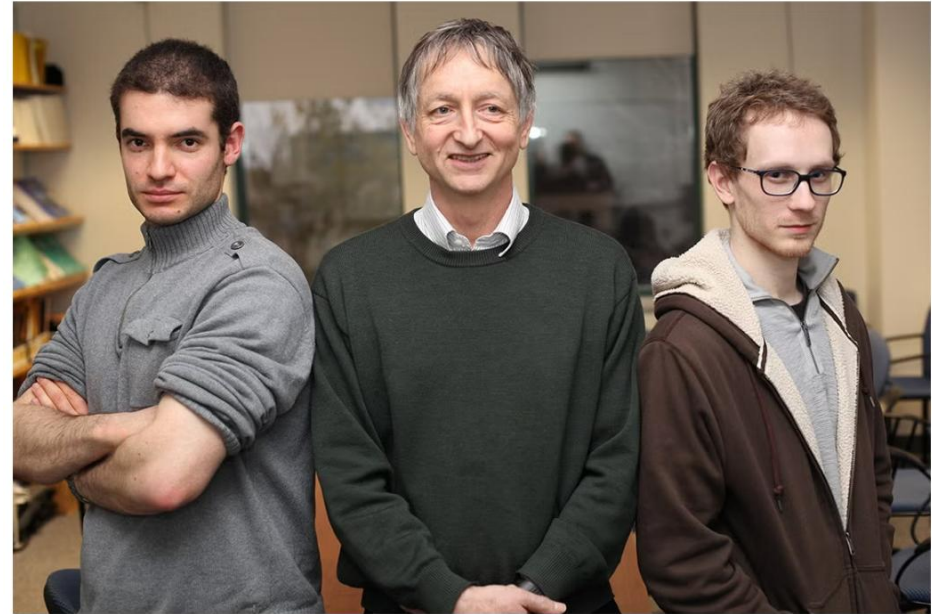
K B A
Z G



신경망 기술 - 딥러닝 (다층 신경망)



전설의 시작: 2012 ImageNet 대회



The 2012 "AlexNet" paper by, from left to right, Ilya Sutskever, Geoffrey Hinton and Alex Krizhevsky, helped launch the deep learning revolution and was ranked eighth on Nature's list (photo by Johnny Guatto)

일리아 수스케버, 제프리 힌튼, 알렉스 크리제브스키

영향

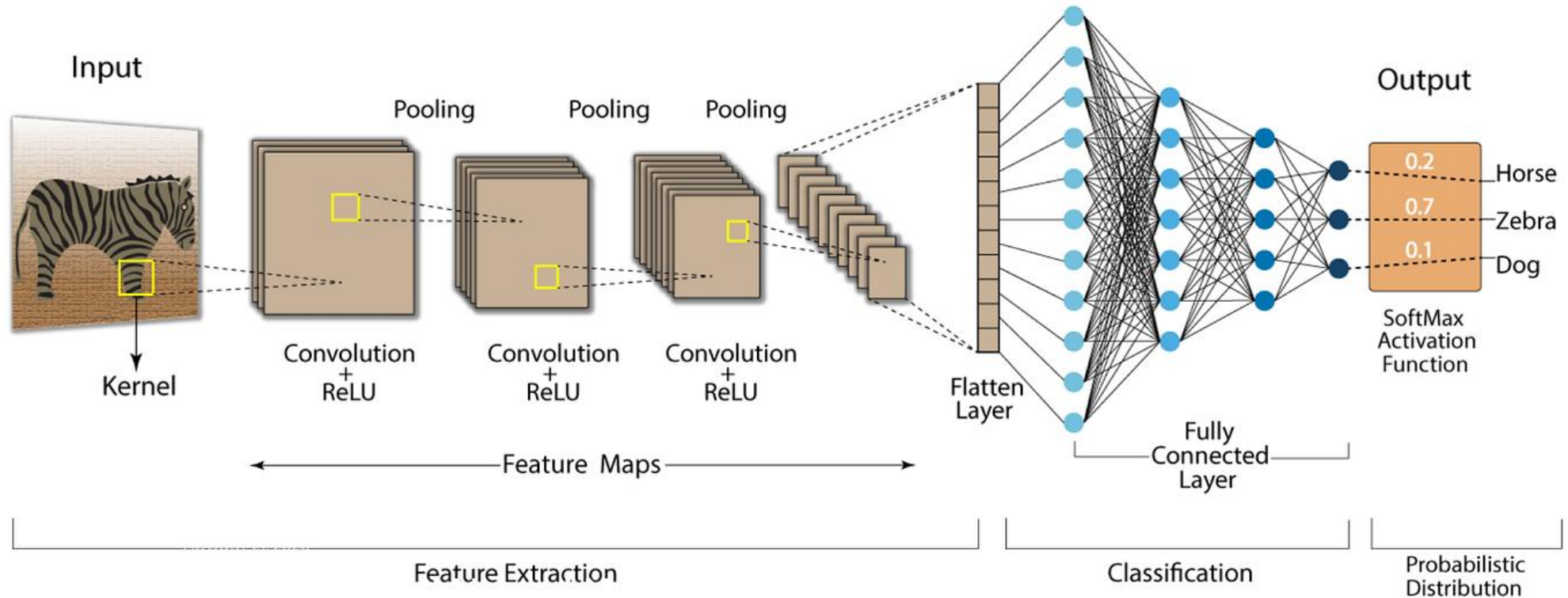
에러율이 26%에서 15%로 감소했습니다.

혁신

신경망 기술을 되살린 결정적인 순간이었습니다.

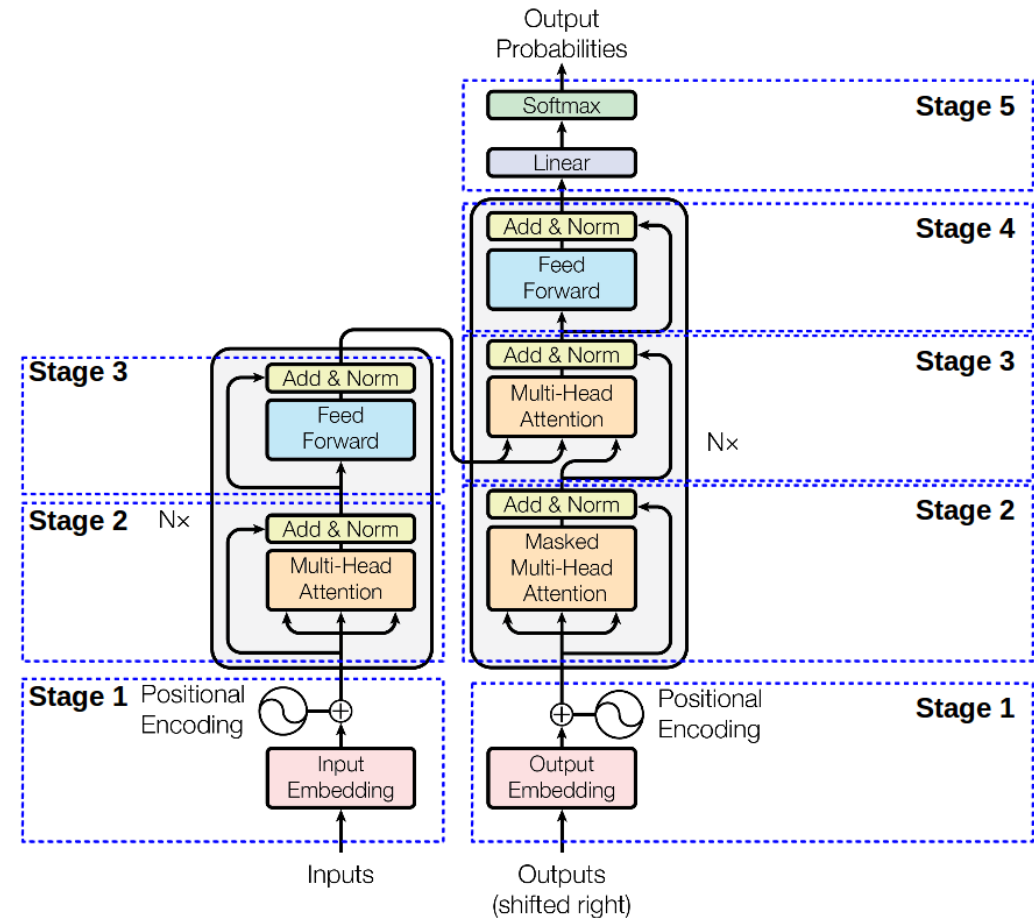
신경망 기술 - 합성곱 신경망 (CNN)

Convolution Neural Network (CNN)



신경망 기술 - 트랜스포머 (Transformer)

- 임베딩 (Embedding)
 - 단어를 벡터로 표현
 - 예) GPT-3에서는 12,288차원
- 어텐션 (Attention)
 - 단어끼리의 어울림 정도
 - 예) 아주 탐스러운 사과



생성형 AI를 사용한 프로그래밍의 진화

- 2021년 코드 자동 완성 GitHub Copilot (GPT-3)
- 2022년 채팅으로 코드 작성 ChatGPT (GPT-3.5)
- 2023년 IDE 내에서 채팅 GitHub Copilot Chat
- 2024년 에이전트 등장 Cursor/GitHub Copilot Agent
Claude Code
- 2025년 바이트 코딩
에이전트 오케스트레이션, 스킬(Skill), MCP, 코딩용 모델 진화
- 2026년 “딸깍”

바이트 코딩은 만능인가?

- 그럴듯하지만 부자연스러운/불완전함
 - Slop
- 비결정적 결과물
 - 같은 프롬프트에도 다른 결과물
- 입력과 출력이 정확하다고 해서 안심할 수 있는가?
 - 쓸데없이 파일을 생성하고 있다면...
- 100% 완성을 위한 비용
 - 80%까지는 쉽지만 나머지 20%에서 시간과 비용 발생
- 갑작스러운 장애에 대한 해결
 - 아무것도 건드린 것이 없는데 장애가 발생한다면?
- 유지보수
 - 비개발자의 한계: "또 공부를 해야 해?"
- 기술 부채
 - 기술을 모르는 채 진행하면 나중에 어려움을 겪을 수 있음
- 이해 부채
 - 코드를 이해하지 못하는 채 진행하면 나중에 어려움을 겪을 수 있음

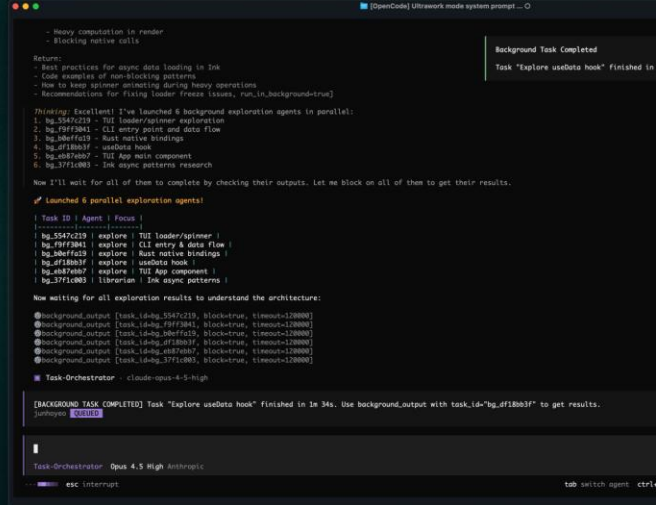
바이브 코딩은 생산적인가?

- AI가 일하는 동안 나는 무엇을 할 것인가?
 - 예) 7시간짜리 작업
 - 무엇을 하려 했는지 잊어버릴 수도 (집중력 감소)
 - 결과물이 마음에 들지 않는다면 다시 시작 (비용 및 시간 낭비)
 - 기다리는 동안 다른 작업을 하게 되면 나중에 정신이 없을 수도
- 드라큘라 이펙트
 - 하루에 8시간 동안 바이브 코딩에 집중할 수 있는가?
 - 3시간이 한계

바이트 코딩은 경제적인가?

**OH MY
OPENCODE**

I spent around
\$24,000 in tokens for this

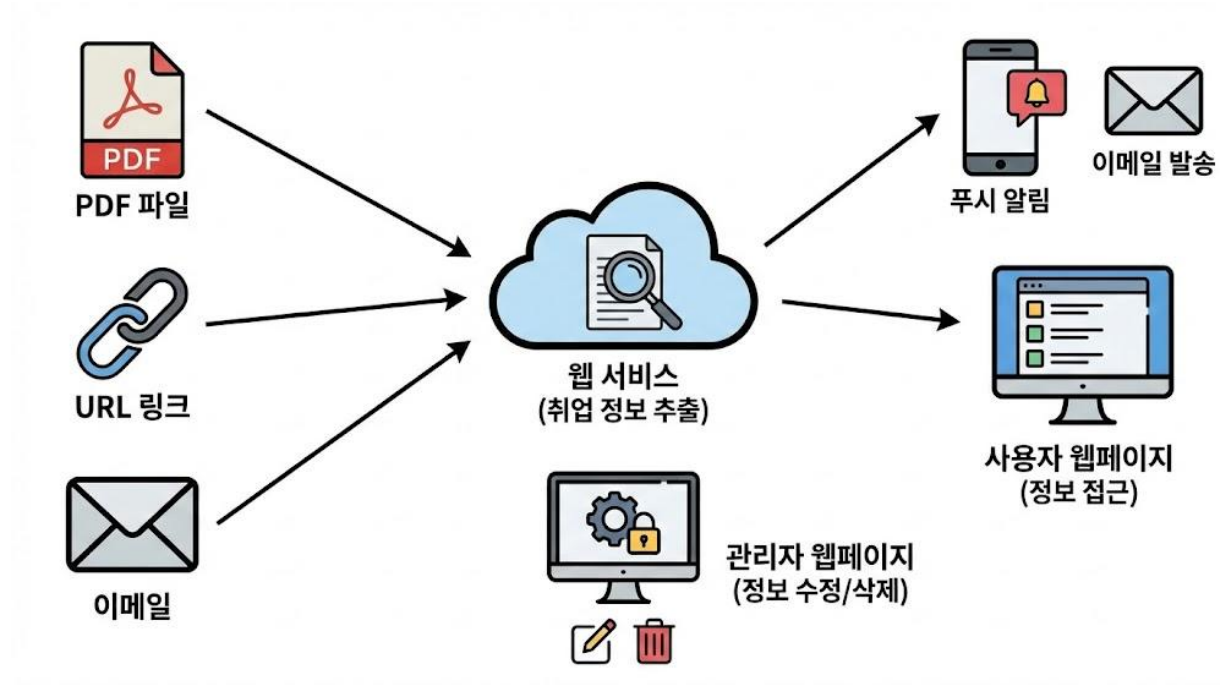


바이브코딩 과목 소개

- 목적
 - 생성형 AI 에이전트를 이용한 프로그래밍 방법을 익힌다.
- 목표
 - 풀스택 웹서비스 개발
- 방식
 - 생성형 AI를 사용한다
 - 코딩 에이전트는 Claude Code
 - API 서비스는 Gemma 4

학과목 프로젝트

- 정보 통합 사이트 (Information Aggregation Site)
- 예) 취업 정보 사이트



사용할 도구

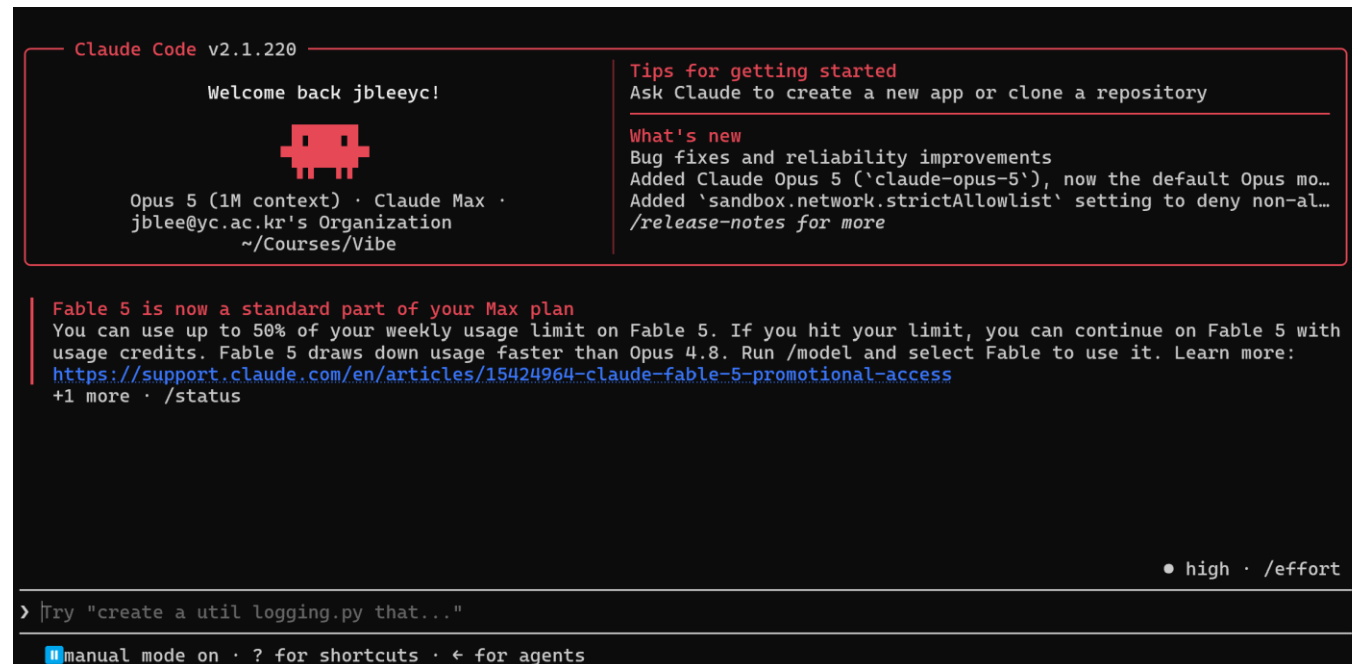
- 운영체제: Ubuntu (Windows 11 WSL2)
- 코딩 에이전트: Claude Code
- 편집기: VS Code
- 터미널: 기본 터미널 또는 Ghostty, Tmux, Orca
- 버전관리: Git/GitHub
- 프레임워크
 - Spring Boot : Java SDK 21 필요
 - React : node.js 24 필요
- 테스트 도구
 - Postman/Newman
 - Playwright

클로드 코드 사용하기

- 설치

```
curl -fsSL https://claude.ai/install.sh | bash
```

- 시작



```
Claude Code v2.1.220

Welcome back jbleeyc!

Opus 5 (1M context) · Claude Max ·
jblee@yc.ac.kr's Organization
~/Courses/Vibe

Tips for getting started
Ask Claude to create a new app or clone a repository

What's new
Bug fixes and reliability improvements
Added Claude Opus 5 ('claude-opus-5'), now the default Opus mo...
Added 'sandbox.network.strictAllowlist' setting to deny non-al...
/release-notes for more

Fable 5 is now a standard part of your Max plan
You can use up to 50% of your weekly usage limit on Fable 5. If you hit your limit, you can continue on Fable 5 with
usage credits. Fable 5 draws down usage faster than Opus 4.8. Run /model and select Fable to use it. Learn more:
https://support.claude.com/en/articles/15424964-claude-fable-5-promotional-access
+1 more · /status

● high · /effort

> Try "create a util logging.py that..."

manual mode on · ? for shortcuts · ← for agents
```

클로드 코드 로그인하기

- 명령어 /login

Login

Claude Code can be used with your Claude subscription or billed based on API usage through your Console account.

Select login method:

- > 1. Claude account with subscription · Pro, Max, Team, or Enterprise
- 2. Anthropic Console account · API usage billing
- 3. 3rd-party platform · Amazon Bedrock, Microsoft Foundry, or Vertex AI

Esc to cancel

클로드 코드 사용량 보기

- 명령어 `/usage`

```
Claude Code v2.1.220
Settings Status Config Usage Stats

Session

Total cost:          $0.0000
Total duration (API): 0s
Total duration (wall): 1m 53s
Total code changes:  0 lines added, 0 lines removed
Usage:               0 input, 0 output, 0 cache read, 0 cache write

Current session
████████████████████████████████████████████████████████████████████████████████ 0% used
Resets 4:30pm (Asia/Seoul)

Current week (all models)
████████████████████████████████████████████████████████████████████████████████ 8% used
Resets Jul 28, 4am (Asia/Seoul)
+50% weekly limits promo through Aug 19 · clau.de/cc-50-promo

Current week (Fable)
████████████████████████████████████████████████████████████████████████████████ 6% used
Resets Jul 28, 4am (Asia/Seoul)

Usage credits
Usage credits are off · /usage-credits to turn them on

Esc to cancel
```

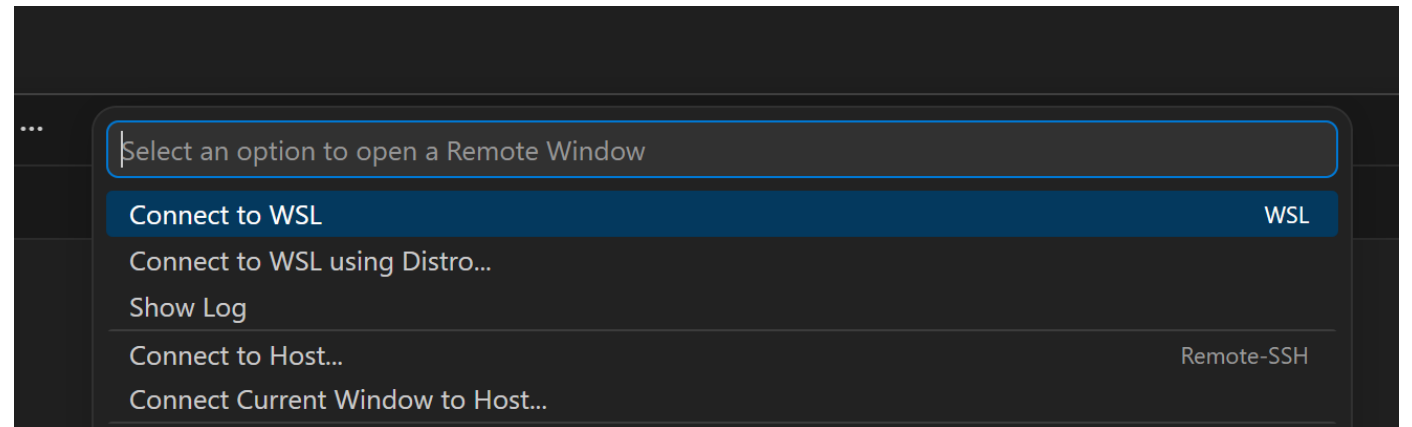
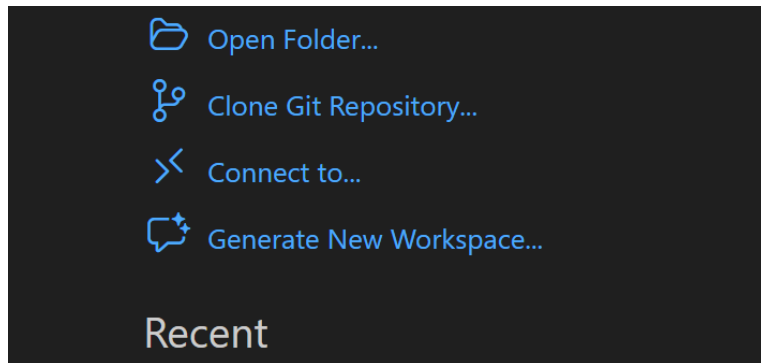
터미널 화면 단순화하기

- <user home directory>/.claude/settings.json

```
{
  "enabledPlugins": {
    "superpowers@claude-plugins-official": true
  },
  "tui": "fullscreen",
  "theme": "dark-ansi",
  "editorMode": "normal",
  "env": {
    "CLAUDE_CODE_HIDE_CWD": "1",
    "CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC": "1"
  }
}
```

VS Code와 WSL2

- WSL2에 있는 소스코드를 윈도우11에 설치된 VS Code로 접근 가능하다
 - "Connect to WSL" 사용



휴식

Vibe coding vs Agentic Engineering

비교 항목	바이브 코딩 (Vibe Coding)	에이전트 엔지니어링 (Agentic Engineering)
핵심 접근법	직관, 감각, 말발(자연어 프롬프트)	아키텍처 설계, 가이드레일, 제약 조건 설정
인간의 역할	결과물을 감상하고 지시하는 디렉터	AI 워크플로우를 통제하는 시스템 설계자
주요 대상	단일 LLM / AI 코딩 어시스턴트	멀티 에이전트(Multi-agent) 시스템, 파이프라인
적합한 프로젝트	프로토타입, 해커톤, 1인 개발, MVP	엔터프라이즈, 대규모 상용 서비스(Production)
품질 관리	인간의 수동 테스트 및 눈대중 검수	자동화된 품질 게이트, 린터, 테스트 커버리지 검증

AI 엔지니어링의 4가지 레이어



랄프 루프 (Ralph Loop)

- 자율형 AI 코딩 자동화 기법 중 하나
- “될 때까지 계속 해!”

```
while ;; do cat PROMPT.md | claude-code ; done.
```

- 반복되는 작업에서 컨텍스트가 부족해질 수 있으므로 메모리를 사용
 - 즉, 중간 결과를 파일에 저장
- Oh-My-OpenCode에서는 시지프스(Sisyphus)라고 불림
- Claude Code나 Codex CLI에는 /goal 명령어가 있음

Git worktree

- Git에서 소스코드는 '워킹 디렉토리(working directory)'에 둔다.
 - 수정 이력은 '저장소(repository)'에 둔다.
 - 개발자가 만지작 거리는 코드는 '워킹 디렉토리'에 존재하는 브랜치이다.
 - 한 개발자가 로컬에서 여러 개의 브랜치를 동시에 편집할 수 없다.
-
- 멀티 에이전트에서는 동시에 여러 벌의 코드가 필요하다.
 - Git worktree는 여러 벌의 '워킹 디렉토리'가 가능하게 해 준다.
 - `$ git worktree add <path> <branch-name>`
 - '워킹 디렉토리'는 여러 개이지만 '저장소(repository)'는 여전히 하나이다.

Prompt and Pray loop

- "xxx를 만들어줘"
- "제발"
- "다시 만들어줘"
- "제발"
- "다시 만들어줘"
- "제발"
- "제발"
- ...

AI 도구의 계획 모드 (Plan mode)

- No “Prompt and Pray” – 코딩하기 전에 계획하라
 - Claude Code, Gemini Cli 등에서 지원
 - Plan mode 사용 중에는 코드를 생성하지 않음
-
- Plan mode가 “요구사항 도출/분석/명세”단계를 대신할 수 있을까?

사양 중심 개발

- Spec Driven Development

“사양 중심 개발”이란, 구현 전에 API 사양서나 요구사항 정의서를 먼저 작성하고 이를 “단일 진실 공급원(Single Source Of Truth)”으로 삼아 설계, 개발, 테스트를 진행하는 방법론입니다.

예) Spec-kit, OpenSpec, Kiro 등등

스킬 vs 플러그인

- 스킬

- AI 에이전트가 참조하는 일종의 함수
- 사용자의 프롬프트/컨텍스트를 보고 적합한 스킬을 찾아 실행한다
- 스킬 자체도 사실은 프롬프트이다

- 플러그인

- 여러 개의 스킬들을 한번에 배포하기 위한 패키지
- 과거에는 명령어(command)도 포함했으나 최근에는 명령어와 스킬을 통합하는 추세이다

클로드 코드의 기본 스킬들

- "claude-plugins-official"에 포함되어 있다
 - feature-dev — 코드베이스 탐색 → 설계 → 구현을 한 흐름으로 묶어줍니다.
 - commit-commands — 커밋/PR 메시지 자동화
 - security-guidance — 파일 편집마다 흑으로 9가지 취약점 패턴을 감시합니다
 - language servers (LSP) — 언어와 관련된 정의 점프.타입 체크가 확실히 좋아 집니다.
 - Context7 — 버전별 최신 문서와 코드 예제를 소스 저장소에서 직접 컨텍스트로 끌어옵니다.
 - frontend-design — 프론트 작업이 있다면. AI가 만든 화면이 다 똑같아 보이는 문제를 줄여줍니다.

스킬의 적절한 개수는?

- 너무 많은 스킬은 오히려 코딩 결과물의 품질을 떨어뜨린다고 함
- 프로젝트에 고유한 규칙은 직접 스킬을 만들거나 CLAUDE.md 파일에 저장하는 것이 바람직하다

안드레이 카파시의 제안

- '바이브 코딩' 용어를 만든 사람
- OpenAI 창업자 중 한 사람
- 2026년 1월: AI가 코드를 작성할 때 지켜야 할 원칙 제안
 - 먼저 생각하라
 - 단순하게 풀어라
 - 필요한 부분만 바꾸라
 - 목표를 분명히 하고 실행하라
- 이 제안을 '헌법' 또는 'Skill'로 지정
 - CLAUDE.md, AGENTS.md, GEMINI.md

Superpowers (1)

- 워크플로 기반의 에이전트 엔지니어링용 프레임워크
 - URL: <https://github.com/obra/superpowers>
- 클로드 코드용 설치

```
/plugin install superpowers@claude-plugins-official
```

```
/superpowers:brainstorming
```

```
/superpowers:writing-plans
```

```
(superpowers) You MUST use this before any creative work - creating  
features, building components, adding functionality, or modifying b...  
(superpowers) Use when you have a spec or requirements for a  
multi-step task, before touching code
```

```
> /superpowers|
```

```
⏏ manual mode on
```

Superpowers (2)

일반적인 개발 워크플로 (greenfield)

- superpowers:brainstorming (기획)
 - 스펙 문서 작성 (design.md)
 - 예) superpowers:brainstorming Next.js와 Tailwind를 사용해서 개인 자산 관리 대시보드 프로토타입을 만들고 싶어. 내 요구사항과 제약 조건을 명확히 하기 위해 질문을 시작해줘.
- superpowers:using-git-worktrees (작업공간 분리)
- superpowers:writing-plans (할 일 정리)
 - 작업 계획서 작성 (plan.md)
- superpowers:executing-plans (구현)
 - Subagent-driven-development 명령도 있음
 - TDD 방식으로 동작

Superpowers (3)

이미 있는 코드를 사용할 때 (brownfield)

- superpowers:brainstorming (분석)
 - 스펙 문서 작성 (design.md)
 - 예) /superpowers:brainstorm 현재 레거시 API 코드의 병목 구간을 분석하고, 성능을 40배 이상 개선하기 위한 아키텍처 대안을 함께 브레인스토밍해줘.
- superpowers:writing-plans (할 일 정리)
 - 작업 계획서 작성 (plan.md)
 - 예) /superpowers:write-plan 기존 user 도메인 코드를 분석해서, 새로운 소셜 로그인(OAuth) 기능을 추가하기 위한 구체적인 구현 계획서를 짜줘.
- superpowers:executing-plans (구현)
- superpowers:refactor (리팩토링)

요구사항 분석 - 과제

- PRD를 만들기
- 이메일로 제출할 것
 - jblee@yc.ac.kr